

# Grundlagen der C-Programmierung

Dr. Henrik Brosenne  
Georg-August Universität Göttingen  
Institut für Informatik

Wintersemester 2024/25

## Beispielprogramme

### Formatierte Ein-/Ausgabe

Formatierte Ausgabe

Formatierte Eingabe

Schleifen

Wertzuweisung

Zeilenorientierte Verarbeitung

Lesen bis zum Ende

Alternativen

Felder

Kommandozeile

Aufgaben

# Ein-/Ausgabe

C selbst kennt zwar keine Anweisungen zur Ein-/Ausgabe.

Die Standardbibliothek stellt entsprechende Funktionen zur Verfügung.

Durch eine Präprozessor-Direktive am Anfang eines Quelltextes können die Deklarationen dieser Funktionen in dem Quelltextes eingefügt werden, sodass die Funktionen benutzt werden können.

```
#include <stdio.h>
```

Die Ausgabefunktion **printf** schreibt immer auf das **Standardausgabegerät**, in der Regel ist das der Bildschirm.

Ihr Pendant für die Eingabe ist die Funktion **scanf**, die stets vom **Standardeingabegerät** liest, in der Regel ist das die Tastatur.

Streng genommen sind **scanf** und **printf** Funktionen zur **formatierte Ein-/Ausgabe**.

Im Zuge der Ein-/Ausgabe erfolgt eine Umwandlung zwischen **interner** (binärer) und **externer** Darstellung.

# Beispiel

```
1  /*****
2   * formatted input/output
3   *****/
4  #include <stdio.h>
5
6  /*=====*/
7  int main() {
8      int value;
9
10     scanf("%d", &value);        // read value
11     printf("%d", value);        // write value
12
13     return 0;
14 }
```

Der **Formatstring** "%d" bei beiden Funktionen zeigen an, dass ein ganzzahliger Dezimalwert mit dem Typ **signed int** zu lesen bzw. zu schreiben ist.

Der **Operator &** bei **scanf** zeigt an, dass ein Wert in der Variablen zu speichern ist (Adressoperator).

# Ablauf

---

```
10     scanf("%d", &value);           // read value
11     printf("%d", value);           // write value
```

---

Ablauf von Übertragung und Umwandlung.

- **scanf** fordert Eingabe von der Standardeingabe an, geliefert werden (Ordnungszahlen von) Zeichen.
- Die gelesene Zeichenfolge soll in einer **int**-Variablen gespeichert werden. Die Zeichen werden **interpretiert** und in die entsprechende binäre Darstellung umgewandelt.

Wie viele Bits erzeugt werden hängt davon ab, wieviel Bits zur Speicherung von **int**-Werten auf dieser Rechnerarchitektur verwendet werden und nicht von der Größe des Wertes.

- **printf** soll einen **int**-Wert, der in Binärdarstellung vorliegt, ausgeben. Die Standardausgabe erwartet Zeichen, deshalb wird die Binärdarstellung in Zeichen umgewandelt.
- Die Zeichen werden auf die Standardausgabe übertragen.

Tatsächlich ist der Ablauf sehr viel komplizierter.

# Formatierte Ausgabe, `printf`

Funktionen erhalten grundsätzlich extrem wenig Information über die **Argumente** aus dem Aufruf, entweder den **Wert** oder die **Speicheradresse**.

Der Typ der Argumente kann beim Aufruf nicht mitgeteilt werden.

Funktionen kennen die Typen ihrer Argumente meist selbst.

## Beispiel

Eine Funktion zur Berechnung einer Winkelfunktion kann sinnvollerweise nur eine Gleitkommazahl als Argument erhalten. Man kann für die drei verschiedenen Gleitkommatypen jeweils unterschiedliche Funktionen vorsehen, wobei jede der Varianten den Typ ihres Argumentes genau kennt.

Bei den Funktionen für die Ein-/Ausgabe wäre es extrem lästig, wenn man für jeden Typ eine andere Funktion aufrufen müsste.

Deshalb geht man anders vor. Nur das erste Argument (ein **Formatstring**) ist festgelegt und damit den Funktionen bekannt.

Der Formatstring beschreibt die weiteren Argumente und zwar sowohl die Anzahl als auch die Typen der Argumente.

# Formatbeschreiber (1/2)

Die Argumente werden durch **Formatbeschreiber** dargestellt.

Ein Formatbeschreiber wird mit einem Prozentzeichen (%) eingeleitet, dem ein Kennbuchstabe (eine Kennbuchstaben-Kombination) für den Typ des Wertes folgt.

An der Anzahl der Formatbeschreiber (= Prozentzeichen) im Formatstring erkennt `printf` die Anzahl der noch folgenden Argumente.

Die Anzahl der Formatbeschreiber kann auch Null sein, siehe Hello World.

Anhand der Kennbuchstaben erkennt `printf`, welche Typen die Argumente haben. Dabei erfolgt die Zuordnung zwischen den Formatbeschreibern und den Ausgabewerten linear.

Der Programmierer trägt die Verantwortung dafür, dass die Formatbeschreiber in Anzahl und Typen den Ausgabewerten entsprechen. Verstöße können zu „kuriosen“ Ausgaben führen – oder auch zum Absturz des Programms.

## Formatbeschreiber (2/2)

Es ist nicht für jeden Typ ein anderer Kennbuchstabe oder eine andere Kombination von Kennbuchstaben erforderlich.

Bei der Übergabe werden

- `float`-Werte grundsätzlich in `double` umgewandelt,
- `short`-Werte grundsätzlich in `int` umgewandelt,
- `char`-Werte werden grundsätzlich in `int` umgewandelt.

Der Kennbuchstabe `c` für den Typ `char` bewirkt die Rückumwandlung von `int` nach `char` durch `printf`.



# Kennbuchstaben für Ausgabeformate

|   | "Standard"-Typ | Darstellung                                   |
|---|----------------|-----------------------------------------------|
| i | signed int     | ganzzahlig dezimal                            |
| d |                | ganzzahlig dezimal                            |
| u | unsigned int   | ganzzahlig dezimal                            |
| o |                | ganzzahlig oktal                              |
| x |                | ganzzahlig hexadezimal                        |
| X |                | ganzzahlig hexadezimal                        |
| f | double         | exponentenfrei, mit<br>oder ohne Dezimalpunkt |
| e | double         | halblogarithmisch                             |
| E |                |                                               |
| g | double         | exponentenfrei oder                           |
| G |                | halblogarithmisch                             |
| c | char           | einzelnes Zeichen                             |
| s | char *         | Zeichenfolge                                  |
| p | void *         | Adresse                                       |

Argument mit dem Typ `long`, dem `int`-Kennbuchstabe `l` **voranstellen**.

Argument mit dem Typ `long double`, dem `double`-Kennbuchstaben `L` **voranstellen**.

# Beispiel

Die Anzahl der Zeichen, die zur Darstellung eines Wertes verwendet werden, richtet sich nach Standard-Vorgaben, wenn man nur Kennbuchstaben verwendet. Zeichen im Formatstring, die nicht zu einem Formatbeschreiber gehören, werden unverändert in die Ausgabe übernommen.

## Beispiel

```
int i;  
long l;  
float f;  
double d;  
long double ld;
```

Formatierte Ausgabe.

```
printf ("Ganze Zahlen, dezimal: %d, %ld", i, l);  
printf ("Ganze Zahlen, oktal: %o, %lo", i, l);  
printf ("Gleitkommazahlen: %e, %E", f, d);  
printf ("Gleitkommazahlen: %f, %Lf", f, ld);
```

In allen vier Beispielen erscheint zunächst der Text, dann der Wert der ersten Variablen, dann ein Komma und schließlich der Wert der zweiten Variablen.

Schauen Sie es sich die Ausgabe am Rechner einmal selbst an.

# Formatierte Eingabe, `scanf`

Die Funktion `printf` benötigt nur die **Werte** ihrer Argumente; ob sie Konstanten oder Variablen sind und wo sie ggf. im Speicher stehen, ist gleichgültig.

Den Wert einer Variable liefert der Name der Variable.

Die Funktion `scanf` muss wissen, **wohin** sie den gelesenen Wert schreiben soll, d.h. sie benötigt nicht die Werte ihrer Argumente, sondern deren **Adressen**.

Die Adresse einer Variablen liefert der, dem Namen der Variable vorangestellt, **Adressoperator &**.

Den Adressoperator bei den Argumenten von `scanf` zu vergessen, ist ein „beliebter“ Fehler.

# Adressoperator

Was bewirkt das Vergessen des Adressoperators?

## Beispiel

```
int i = 0;  
scanf ("%d", i);
```

Es wird der **Wert** von `i` anstelle der **Adresse** von `i` übergeben, hier also 0.

Die Funktion `scanf` wird durch den Formatstring informiert, dass ein `int`-Wert zu lesen ist, und unterstellt deshalb, dass das nächste Argument die **Adresse** einer `int`-Variablen ist.

Der Versuch, 0 als Adresse zu verwenden, kann nur ins Chaos führen. Wenn man Glück hat, überschreibt man nur Daten, die man ohnehin nicht mehr braucht. Aber Glück hat man in der Regel nicht.

Das Programm wird mit einem **segmentation fault**, **memory fault** oder **bus error** vom Betriebssystem „gekillt“.

# Formatstring

Durch den Formatstring, der als erster Parameter übergeben wird, muss der Funktion mitgeteilt werden, wie viele Argumente folgen und welche Typen diese besitzen.

Die Verantwortung für die Übereinstimmung von Formatstring und Argumentliste hat der Programmierer.

# Unterschiede zu `printf`

- Eine Typumwandlung ist nicht möglich.

Ist z.B. ein Zeichen zu lesen, so umfasst der Speicherplatz der entsprechenden Variablen nur Platz für genau ein Zeichen, so dass `scanf` nur genau soviel Bits an die übergebene Adresse in den Speicher schreiben darf.

- Ob die Eingabezeichen für ganze Zahlen als dezimale, oktale oder hexadezimale Darstellung des Wertes zu interpretieren sind, ist wesentlich.

Es gilt  $11_d \neq 11_o \neq 11_h$ .

Ebenfalls wesentlich ist, ob ein ganzzahliger Eingabewert ein Vorzeichen tragen darf oder nicht.

Anders bei Gleitkommazahlen. Welche der drei möglichen Darstellungen bei der Eingabe verwendet wird, ist für die interne Darstellung der Werte bedeutungslos.

- Zeichen im Formatstring, die nicht zu Formatbeschreibern gehören, müssen bei der Eingabe eingetippt werden. An die rufende Funktion werden sie nicht weitergegeben.

Zweckmäßig ist es, die Formatbeschreiber durch ein Leerzeichen (repräsentiert alle white spaces) voneinander zu trennen, andere Zeichen in Eingabeformaten aber nicht zu verwenden.

# Kennbuchstaben für Eingabeformate (1/2)

|   | erwartete Zeichenfolge                       | “Standard”-Typ |
|---|----------------------------------------------|----------------|
| i | ganzzahlig, mit oder ohne Vorzeichen         | signed int *   |
| d | ganzzahlig dezimal, mit oder ohne Vorzeichen |                |
| u | ganzzahlig dezimal                           | unsigned int * |
| o | ganzzahlig oktal                             |                |
| x | ganzzahlig hexadezimal                       |                |
| X | ganzzahlig hexadezimal                       |                |
| f | Gleitkommawert, mit                          | float *        |
| e | oder ohne Vorzeichen,                        |                |
| E | mit oder ohne Dezimal-                       |                |
| g | punkt, mit oder ohne                         |                |
| G | Exponententeil                               |                |
| c | einzelnes Zeichen oder Zeichenfolge          | char *         |
| s | Zeichenfolge                                 | char *         |

# Kennbuchstaben für Eingabeformate (2/2)

Andere Typen müssen durch einen zweiten, **vorangestellten** Kennbuchstaben gekennzeichnet werden.

- h short bei ganzzahligen Werten
- l long bei ganzzahligen Werten
- double bei Gleitkommawerten
- L long double bei Gleitkommawerten

## Beispiel

```
short s;  
int i;  
long l;  
float f;  
double d;  
long double ld;  
  
scanf ("%hd", &s);  
scanf ("%d", &i);  
scanf ("%ld", &l);  
scanf ("%f", &f);  
scanf ("%lf", &d);  
scanf ("%Lf", &ld);
```



## Beispielprogramme

Formatierte Ein-/Ausgabe

**Schleifen**

Wertzuweisung

Zeilenorientierte Verarbeitung

Lesen bis zum Ende

Alternativen

Felder

Kommandozeile

Aufgaben

# Schleifen (1/2)

## Aufgabe

Ein Programm soll zehn (`int`-)Zahlen einlesen und wieder ausgeben.

## Lösungsvorschläge

- Deklaration einer Variable.  
Abwechselnd, zehn Aufrufe von `scanf` und `printf` programmieren.
- Deklaration von 10 Variablen. In jeder der Variablen mit `scanf` einen Wert einlesen. Jeder der Variablen mit `printf` ausgeben.

Dass sind beides keine realistischen Lösungsvorschläge.

## Erweiterte Aufgabe

Ein Programm soll zehn (`int`-)Zahlen lesen und wieder schreiben. Es soll sich ohne größeren Aufwand auf beliebige andere Anzahlen umstellen lassen.

## Schleifen (2/2)

Eine **Schleife** bringt die Lösung.

Die Ausweisungen für Ein- und Ausgabe **einer** Zahl werden in ein Sprachkonstrukt eingebettet, das dafür sorgt, dass sie mehrfach ausgeführt werden.

Höhere Programmiersprachen kennen in der Regel verschiedene Schleifen-Anweisungen, die sich, von Formalismen abgesehen, darin unterscheiden, **wie** die Anzahl der erforderlichen Wiederholungen bestimmt wird.

# while-Schleife

## while-Schleife

```
while (bedingung) {  
    anweisungsfolge  
}
```

Vor jedem Schleifendurchlauf wird die Bedingung *bedingung* ausgewertet.

Ist das Ergebnis **wahr** (d.h. ungleich 0), so wird die Anweisungsfolge der Schleife ausgeführt.

Ist das Ergebnis dagegen **falsch** (d.h. gleich 0), so wird zu der Anweisung übergegangen, die dem **Schleifenrumpf** folgt, d.h. zu der ersten Anweisung, die nicht mehr zur Anweisungsfolge der Schleife gehört.

Damit kann die gestellte Aufgabe leicht realisieren werden.

Um die erweiterte Aufgabenstellung zu erfüllen, wird die Anzahl der Zahlen als benannte Konstante festgelegt

# Wiederholt Zahlen einlesen und ausgeben

```
1  #include <stdio.h>
2
3  #define N 10                                // iterations
4
5  //=====
6  int main()
7  {
8      int i = N, value;
9
10     printf("Enter values:\n");
11
12     while (i) {                                // loop
13         scanf("%d", &value);                    // read value
14         printf("%d", value);                    // write value
15         i = i - 1;                                // decrement iterations
16     }
17     return 0;
18 }
```

Soll eine andere Anzahl von Zahlen gelesen und geschrieben werden, so muss nur der Wert in der `#define`-Direktive abgeändert und das Programm neu übersetzt und gebunden werden.

## Beispielprogramme

Formatierte Ein-/Ausgabe

Schleifen

**Wertzuweisung**

Zeilenorientierte Verarbeitung

Lesen bis zum Ende

Alternativen

Felder

Kommandozeile

Aufgaben

# Wertzuweisung

Auf die **Wertzuweisung**, bis jetzt stillschweigend verwendet, muss noch kurz eingegangen werden.

```
i = i - 1;
```

Die Wertzuweisung bewirkt, dass der Ausdruck rechts vom Gleichheitszeichen berechnet und der resultierende Wert in der Variablen links vom Gleichheitszeichen gespeichert wird.

Die Wertzuweisung ist keine Gleichung im Sinne der Mathematik.

Der Ausdruck auf der rechten Seite kann wie eine mathematische Formel aus mehreren Operanden und Operatoren bestehen.

Als Operatoren stehen (zunächst) die mathematischen Operatoren zur Verfügung.

- + Addition
- Subtraktion
- \* Multiplikation
- / (ganzzahlige) Division
- % Rest der ganzzahligen Division

## Beispielprogramme

Formatierte Ein-/Ausgabe

Schleifen

Wertzuweisung

**Zeilenorientierte Verarbeitung**

Lesen bis zum Ende

Alternativen

Felder

Kommandozeile

Aufgaben



# Zeilenorientierte Verarbeitung (1/2)

Probiert Sie das letzte Programm einmal aus und geben Sie mehrere Zahlen durch Leerzeichen getrennt auf einer Zeile ein.

Damit erzeugt man ziemliches Durcheinander auf dem Bildschirm. Warum?

Der Bildschirm hat eine Doppelfunktion.

Einerseits werden auf ihm die Tastatur-Anschläge protokolliert; andererseits dient er als Standardausgabe für die Programme.

Der Hintergrund für das Durcheinander ist die **gepufferte Eingabe**.

Die Tasten, die man auf der Tastatur anschlägt werden zwar sofort dem Bildschirm protokolliert (**geecho**).

Einem Programm werden die Zeichen allerdings erst zur Verfügung gestellt, wenn die ENTER-Taste gedrückt wird (was als Echo einen Zeilenvorschub ergibt).

## Zeilenorientierte Verarbeitung (2/2)

Das Beispielprogramm bekommt die Zeile erst nach dem Drücken der ENTER-Taste.

Enthält die Zeile, die es bekommt, mehrere Zahlen, so liest und schreibt es diese, ohne neuerliche Eingabe zu erwarten, auch wieder in eine Zeile.

Da das Ausgabeformat keinerlei Trennung zwischen den Werten vorsieht, erscheinen die Werte bei der Ausgabe auch unmittelbar hintereinander.

Ist die Maximalzahl der zu kopierenden Zahlen noch nicht erreicht, bleibt die Ausgabe direkt hinter dem letzten geschriebenen Wert stehen – jetzt wird wieder Eingabe erwartet.

Übersichtlicher wird es, wenn man jeden Wert in eine eigene Zeile ausgibt und den Benutzer bittet, jeden Eingabewert mit ENTER abzuschließen. Außerdem kann man dem Benutzer noch Hinweise zur korrekten Eingabe geben.

# Zahlen kopieren mit Vorschüben

```
1  #include <stdio.h>
2  #define N 10                                // iterations
3
4  //=====
5  int main ()
6  {
7      int i = N, value;
8
9      printf("Enter one value per line please.\n");
10
11     while (i) {                                // loop
12         printf("%d values remaining: ", i);
13         scanf("%d", &value);                    // read value
14         printf("%d\n", value);                  // write value
15         i = i - 1;                              // decrement iterations
16     }
17     return 0;
18 }
```

Eine Möglichkeit, den Benutzer dazu zu zwingen, pro Zeile nur einen Wert einzugeben, haben wir derzeit noch nicht.

## Beispielprogramme

Formatierte Ein-/Ausgabe

Schleifen

Wertzuweisung

Zeilenorientierte Verarbeitung

**Lesen bis zum Ende**

Alternativen

Felder

Kommandozeile

Aufgaben

# Lesen bis zum Ende

## Modifizierte Aufgabe

Das Programm soll so lange `int`-Zahlen lesen und wieder schreiben, wie der Benutzer das wünscht.

In der Terminologie der Datenverarbeitung. Das Programm soll so lange `int`-Zahlen lesen und wieder schreiben, bis das Ende der Eingabedatei erreicht ist.

## Fragen

- 1 Wie kann der Benutzer anzeigen, dass die Datei zu Ende ist?
- 2 Wie erfährt das Programm diese Information?

Antwort auf die erste Frage.

Tippt der Benutzer unter UNIX C-d, so wird das vom Betriebssystem als Dateiende interpretiert.

Antwort die zweite Frage.

Der Funktionswert der verwendeten Funktionen `scanf` liefert die Antwort.

# End of File (EOF)

`scanf` liefert als Funktionswert die Anzahl der gelesenen und erfolgreich umgewandelten Werte; tritt jedoch bereits **vor** dem Lesen und der Umwandlung des ersten Wertes ein Fehler ein (und das Dateiende zählt in diesem Sinne als Fehler), so ist der Funktionswert **EOF** (**End Of File**).

**EOF** ist ein negativer, ganzzahliger Wert, denn alle anderen möglichen Funktionswerte sind ganzzahlig und nichtnegativ.

Häufig ist der Wert  $-1$ , aber das wird vom Standard nicht vorgeschrieben. Die benannte Konstante **EOF** ist wie `scanf` und `printf` in `stdio.h` deklariert.

Wie kann man den Funktionswert nutzen?

Zur Verfügung steht, neben anderen, der **Vergleichsoperator** `!=` (**ungleich**), mit dem wir den Funktionswert mit der Konstanten **EOF** vergleichen können.

Das Resultat ist **wahr** bzw. **falsch**, je nachdem, ob die Relation des Operators zwischen den beiden Operanden besteht oder nicht.

# Lesen bis zum Ende der Eingabe

```
1  /*****
2   * read up to end of file
3   *****/
4  #include <stdio.h>
5
6  //=====
7  int main() {
8      int value, count = 0;
9
10     printf("Enter values:\n");
11
12     while (scanf("%d", &value) != EOF) {          // read value
13         count = count + 1;
14         printf("Value %d = %d\n", count, value);
15     }
16     return 0;
17 }
```

# Anmerkungen (1/2)

Der Funktionswert von `scanf` erlaubt auch die Erkennung von (anderen) Fehler bei der Eingabe.

Ist der Funktionswert kleiner als die Anzahl der Formatbezeichner (und der Variablen), dann ist etwas schief gelaufen.

Sollen z.B. 10 Werte gelesen werden und der Funktionswert ist 8, so kann man daraus rückschließen, dass beim Lesen des neunten Wertes ein etwas schief gegangen ist.

**EOF** wird nur dann geliefert, wenn bereits **vor** dem Lesen des ersten Wertes etwas schief geht. Tritt **beim** Lesen des ersten Wertes ein Fehler ein, so ist der Funktionswert 0. Statt mit **EOF** sollte man den Funktionswert von `scanf` in unserem Beispiel besser mit 1 vergleichen (`scanf (...) == 1`).

## Beispiele

`scanf` soll einen `int`-Wert einlesen.

- Eingabe `z` erzeugt Rückgabewert 0 und undefinierten gelesenen Wert.
- Eingabe `11s` erzeugt Rückgabewert 1 und gelesenen Wert 11.
- Eingabe `5.3` erzeugt Rückgabewert 1 und gelesenen Wert 5.

Der nicht ausgewertete Rest der Eingabe (`z`, `s`, `.3`) verbleibt im Eingabepuffer.



## Anmerkungen (2/2)

Auch `printf` liefert einen wohldefinierten Funktionswert.

Der Standard schreibt vor:

- Bei korrektem Abschluss der Ausgabe ist der Funktionswert die Anzahl der geschriebenen Zeichen.
- Bei fehlerhaftem Abschluss ist der Funktionswert negativ. Den Implementatoren ist es freigestellt, unterschiedliche Fehler durch unterschiedliche Werte zu kennzeichnen.

## Beispielprogramme

Formatierte Ein-/Ausgabe

Schleifen

Wertzuweisung

Zeilenorientierte Verarbeitung

Lesen bis zum Ende

## Alternativen

Felder

Kommandozeile

Aufgaben

# Alternativen

Neben der Bildung von Schleifen gibt es eine zweite grundlegende Programmiertechnik, nämlich die **Auswahl aus Alternativen**.

Realisiert wird sie durch **Verzweigungsanweisungen**.

Bei einer Schleife wird der Schleifenrumpf (eine Folge von Anweisungen) mehrfach oder ggf. auch garnicht ausgeführt

Mit Verzweigungsanweisungen kann man erreichen, dass in Abhängigkeit von bestimmten Bedingungen, aus verschiedenen Anweisungsfolgen eine ausgewählt und ausgeführt wird.

# if-Anweisung

Die wichtigste Verzweigungsanweisung in C ist die `if`-Anweisung.

```
if ( bedingung ) {  
    anweisungsfolge_if  
}  
[ else {  
    anweisungsfolge_else  
} ]
```

Ergibt die Auswertung von *bedingung* den Wert **wahr** wird die *anweisungsfolge\_if* ausgeführt.

Ergibt die Auswertung von *bedingung* den Wert **falsch** und ist der optionale **else**-Teil vorhanden, wird die *anweisungsfolge\_else* ausgeführt.

## Bemerkung

Besteht die Anweisungsfolge nur aus einer Anweisung, kann auf die umschließenden geschweiften Klammern verzichtet werden (gilt auch bei Schleifen).

# Vergleichsoperatoren

|    |                |
|----|----------------|
| == | gleich         |
| != | ungleich       |
| <  | kleiner        |
| <= | kleiner gleich |
| >  | größer         |
| >= | größer gleich  |

Vorsicht beim Test auf Gleichheit. Schreibt man nur ein Gleichheitszeichen (=), wird das vom Compiler akzeptiert, aber die Bedeutung eine **vollständig** andere.

## Aufgabe

Der Benutzer soll eine positive ganze Zahl eingeben; nicht positive Zahlen sollen zurückgewiesen werden.

Eine Aufgabe, die in der Praxis sicher nur als Bestandteil umfangreicherer Aufgaben vorkommt.

# Auswahl aus Alternativen mit if und else

```
1  #include <stdio.h>
2
3  int main() {
4      int value = 0;
5      while (value <= 0) {
6          printf("Enter a positiv value: ");
7          if (scanf("%d", &value) != 1) {                // read value
8              printf("Input error\n");                  // error handling
9              return 1;
10         }
11         else {
12             if (wert <= 0) {                            // input ok?
13                 printf("Value not positiv - try again\n"); // no
14             }
15             else {
16                 printf("ok\n");                          // yes
17             }
18         }
19     }
20     return 0;
21 }
```

Schleifen und Verzweigungsanweisungen dürfen **geschachtelt** werden.

## Beispielprogramme

Formatierte Ein-/Ausgabe

Schleifen

Wertzuweisung

Zeilenorientierte Verarbeitung

Lesen bis zum Ende

Alternativen

**Felder**

Kommandozeile

Aufgaben



In mathematische Formeln werden oft Vektoren oder Matrizen verwendet; in anderen Problemkreisen hat man häufig Tabellen.

Programmiersprachen stellen entsprechende Datenstrukturen unter dem Oberbegriff **Felder (array)** zur Verfügung.

Eindimensionale Felder bezeichnet man als **Vektoren** und zweidimensionale Felder werden als **Matrizen** bezeichnet.

Das Wesen eines Feldes ist, dass es aus einer **festen Anzahl** von **Elementen (Komponenten)** besteht, die sämtlich **denselben Typ** besitzen.

# Vektoren deklarieren

In C deklariert man Vektoren, indem man dem Namen des Feldes in der Variablendeklaration die gewünschte Komponentenzahl nachstellt, eingeschlossen in eckige Klammern.

```
double v[5];
```

oder besser

```
#define LAENGE 5  
...  
double v[LAENGE];
```

Diese Deklaration besagt, *v* ist ein Vektor (eindimensionales Feld) mit 5 Komponenten.

ANSI-C fordert Konstanten für die Dimensionierung von Felder. Seit C99 stehen lokal auch Felder variabler Größe (siehe *Variable Length Arrays*) zu Verfügung.

# Komponenten ansprechen

Ansprechen kann man einzelne Komponenten eines Vektors, indem man seinem Namen den **Index** der entsprechenden Komponente nachstellt, erneut eingeschlossen in eckige Klammern.

**Achtung.** Die Numerierung der Komponenten beginnt **immer** bei 0, daraus folgt der höchste (erlaubte) Index ist um 1 kleiner als die Länge des Vektors.

Im Beispiel hat  $v$  also die Komponenten  $v[0]$ ,  $v[1]$ ,  $\dots$ ,  $v[4]$ .

Die Indizes der Komponenten müssen keine Konstanten sein. Erlaubt sind (u.a.) auch Variablen. Diese Variablen müssen einen ganzzahligen Typ besitzen und ihre Werte müssen innerhalb des Indexbereiches des Vektors liegen.

# Aufgabe

## Aufgabe

- Einlesen zweier `double`-Vektoren,
- Berechnung des Skalarprodukt der beiden Vektoren,
- Ausgabe des Resultats.

# Berechnen eines Skalarproduktes (1/2)

```
1  #include <stdio.h>
2  #define N 5
3
4  //=====
5  int main() {
6      double v[N], w[N], prod;          // declaration of variables
7      int i;
8
9      printf("Enter first vector\n");    // read 1st vector
10     i = 0;
11     while (i < N) {
12         scanf ("%lf", &v[i]);
13         i = i + 1;
14     }
15
16     printf("Enter second vector\n");    // read 2nd vector
17     i = 0;
18     while (i < N) {
19         scanf ("%lf", &w[i]);
20         i = i + 1;
21     }
22
```

# Berechnen eines Skalarproduktes (1/2)

---

```
23 // compute scalar product -----
24 i = 0;
25 prod = 0;
26 while (i < N) {
27     prod = prod + v[i] * w[i];
28     i = i + 1;
29 }
30
31 printf("Scalar_\product:_%f\n", prod);
32
33 return 0;
34 }
```

---

## Beispielprogramme

Formatierte Ein-/Ausgabe

Schleifen

Wertzuweisung

Zeilenorientierte Verarbeitung

Lesen bis zum Ende

Alternativen

Felder

**Kommandozeile**

Aufgaben

# Aufgabe

## Aufgabe

- Übernehmen eines Kreisradius (`double`-Wert) von der Kommandozeile,
- Berechnen des Umfangs und der Fläche des Kreises,
- Ausgabe des Resultats.



# Kreis

---

```
1  /*****
2   * command line example
3   *****/
4  #include <stdio.h>
5  #include <stdlib.h>
6
7  //=====
8  int main(int argc, char *argv[]) {
9      double radius = atof(argv[1]);    // interpret command line
10     double pi = 3.14159265359;
11
12     printf("A circle with radius %f has circumference %f"
13           "and area %f.\n", radius, 2*pi*radius, pi*radius*radius);
14
15     return 0;
16 }
```

---

## Beispielprogramme

Formatierte Ein-/Ausgabe

Schleifen

Wertzuweisung

Zeilenorientierte Verarbeitung

Lesen bis zum Ende

Alternativen

Felder

Kommandozeile

**Aufgaben**

# Aufgaben

## Aufgabe

Probieren Sie alle Beispiele aus.